C PROGRAMMING · STUDY GUIDE

Functions

Concept, Interface, Arguments, Recursion & Parameter Passing in C

A complete, exam-oriented reference covering the concept of subprograms, function interfaces, arguments, return statements, local variables, iterative and recursive subprograms, recursion, call by value, call by reference, and the underlying implementation of function calls — with diagrams, comparison tables, solved programs, and quick revision notes.

- University Exams
- Competitive Exams
- Teacher Recruitment Tests
- C Programming

Table of Contents

Follow the topics in order for complete, syllabus-aligned coverage.

01 Concept of a Subprogram

02 The Interface of a Subprogram

03 Role of an Interface

04 Arguments of a Subprogram

05 Kinds of Subprograms Supported by C

06 Return Statement as an Interface

07 Local Variables

08 Default Return Type and the void Type

09 Passing Values Between Functions Through Interfaces

10 Declaration of Function Type

11 Iterative and Recursive Subprograms

12 Recursion

13 Concept of Call by Value

14 Concept of Call by Reference

15 Return Mechanism and Underlying Implementation

16 Similarities and Comparisons

17 Programming Examples

18 Quick Revision

TOPIC 01

Concept of a Subprogram

Definition

DEFINITION

A **subprogram** (also called a **function**) is a self-contained, named block of program statements that performs a specific, well-defined task. It can be invoked (called) from other parts of the program as many times as required, and it may accept input values and/or return a result to the part of the program that called it.

In C, every subprogram is written as a **function**. Every C program itself is built from at least one function, `main()`, which acts as the entry point of execution; large problems are broken down into smaller functions that `main()` and other functions call as needed.

Purpose and Advantages

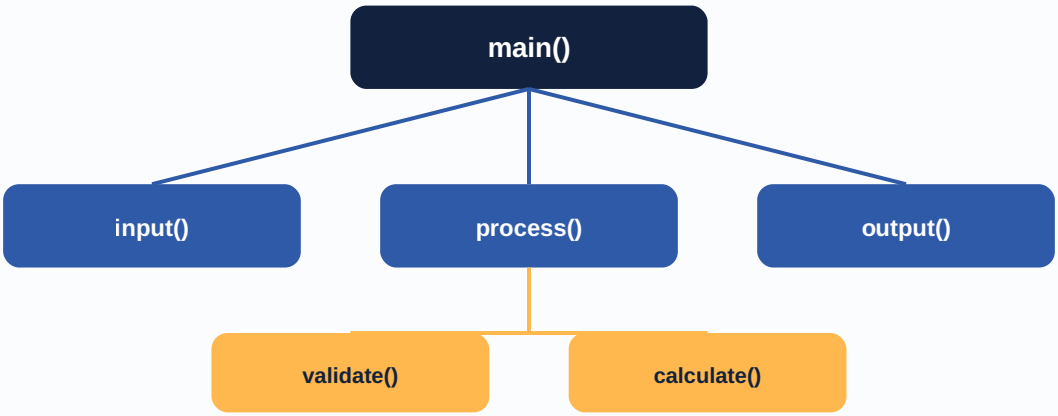
- **Decomposition** — a large, complex problem is broken into smaller, manageable sub-tasks.
- **Reusability** — the same block of code can be called repeatedly from different places without rewriting it.
- **Readability** — meaningful function names make the program self-documenting and easier to follow.
- **Easier debugging and testing** — each function can be tested independently (unit testing).
- **Maintainability** — changes are localized to one function instead of scattered code.
- **Abstraction** — the caller only needs to know *what* a function does, not *how* it does it.
- **Reduced code size** — avoids duplication of identical logic at multiple places.
- **Team development** — different programmers can work on different functions simultaneously.

What does "scattered code" mean?

"Scattered code" means the same logic is copy-pasted or spread out in multiple places in your program instead of living in one function.

Role of Functions in Modular Programming

Modular programming is a design technique in which a program is divided into separate, independent units called **modules** (functions in C), each responsible for one aspect of the program's functionality. Functions are the basic building blocks of modular programming in C: the main problem is split into sub-problems, each sub-problem is implemented as one function, and the functions are combined (through function calls) to solve the complete problem.



A single program (main) divided into smaller subprograms, each doing one task

Diagram: A program decomposed into subprograms (modular programming)

Example — Program without and with a subprogram

```c
// Without a subprogram (repetitive)
int main() {
    int a = 5, b = 6, c;
    c = a * a;   printf("%d\n", c);
    c = b * b;   printf("%d\n", c);
    return 0;
}

// With a subprogram (reusable)
int square(int x) {          // subprogram
    return x * x;
}
int main() {
    printf("%d\n", square(5));
    printf("%d\n", square(6));
    return 0;
}
```

Output: `25 36 25 36`

Exam Tip

In C terminology, "subprogram", "function", and "procedure" are used interchangeably. Unlike some languages, C does not have a separate "procedure" keyword — every subprogram in C is written as a function (a function that does not return a value simply uses `void` as its return type).

CHAPTER SUMMARY

- A subprogram is a named, reusable block of code performing a specific task.
- Functions support decomposition, reusability, readability, and easy debugging.
- Modular programming divides a problem into independent functions that cooperate to solve the whole problem.
- Every C program has at least one function: `main()`.

TOPIC 02

The Interface of a Subprogram

Definition

DEFINITION

The **interface** of a subprogram is the formal specification of how the calling code communicates with the function — it defines the function's **name**, the **type and number of arguments (parameters)** it accepts, and the **type of value it returns**. In C, this interface is expressed through the **function prototype**.

Example — Interface of a function

```
int add(int a, int b);
```

Breakdown: `Name -> add Parameters -> two ints (a, b) Return type-> int`

The interface acts as a "contract" between the function and the code that calls it: it tells the caller exactly what to supply and what to expect back, without exposing the internal implementation details of the function.

Function Prototype

A **function prototype** is a declaration that specifies the function's interface before it is used, usually placed before `main()` or in a header file.

SYNTAX

```
return_type function_name(parameter_type1, parameter_type2, ...);
```

Example — Function prototype

```
int add(int, int);      // prototype: takes two ints, returns an int
float area(float);      // prototype: takes one float, returns a float
void display(int);      // prototype: takes an int, returns nothing
```

Aspect	Interface	Prototype
What it is	The design concept — how the caller and function communicate	The concrete C syntax that expresses that concept
Includes	Name, argument types/order, return type (the contract)	A single declaration statement written in code
Example	Interface of `add` : name `add` , params two ints, returns int	`int add(int a, int b);`
In short	The idea	The code that expresses the idea

Components of a Function Interface

Component	Description
Function name	A unique identifier used to call the function.
Return type	The data type of the value the function sends back to the caller (`void` if nothing is returned).
Parameter list	The type (and optionally name) of each value the function expects to receive.
Function body	Not part of the interface itself — it is the implementation (hidden from the caller).

Working Mechanism

1. The compiler reads the function prototype/declaration and checks every function call against it (this is called **type checking**).
2. When a function is called, control transfers to the function definition matching that interface.

3. Arguments passed by the caller must match the parameter types declared in the interface.
4. The value returned (if any) must match the declared return type.

Example — Calling function vs Called function

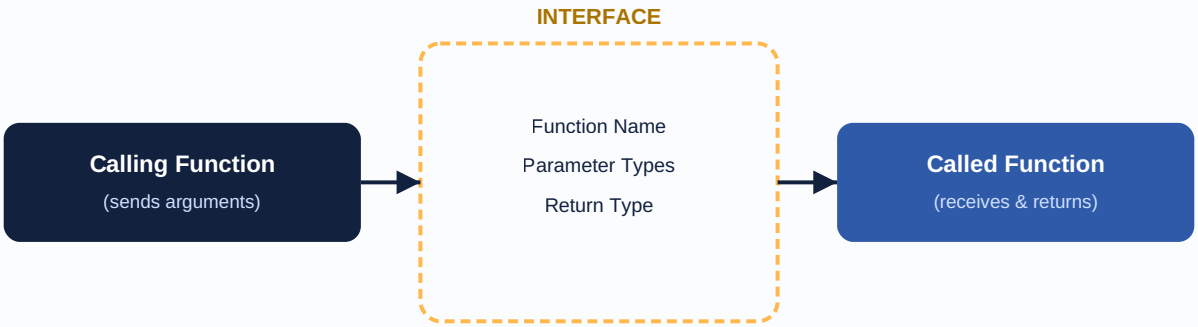
```
int add(int a, int b) {      // <-- CALLED FUNCTION (this one gets invoked)
    return a + b;
}

int main() {                 // <-- CALLING FUNCTION (this one does the calling)
    int result = add(3, 4);  // main() calls add() here
    printf("%d\n", result);
    return 0;
}
```

Explanation: `main()` is the calling function -- it contains the statement that triggers the call. `add()` is the called function -- it is the one being invoked to do the actual work. Once `add()` finishes, control returns to `main()` right after the point where it was called.

Note

"Calling" and "called" describe a *relationship* for a particular call, not a fixed label — the same function can be a calling function in one call and a called function in another, depending on which role it plays.



The interface mediates all communication between caller and function

Diagram: Function interface as a mediator between caller and function

Note

Without a matching prototype, the compiler cannot verify argument types at the call site, which can lead to type-mismatch bugs that are hard to detect. Modern C compilers require (or strongly recommend) prototypes for this reason.

CHAPTER SUMMARY

- The interface defines *what* a function needs and returns — not *how* it works internally.
- It consists of the function name, parameter types, and return type — expressed as a prototype.
- The compiler uses the prototype to type-check every call to the function.

TOPIC 03

Role of an Interface

Importance of Function Interfaces

- **Separation of concerns** — the caller does not need to know the internal logic of the function, only need to know its interface.
- **Type safety** — the compiler validates that arguments passed, match the expected types, catching errors at compile time.
- **Documentation** — the prototype itself documents what a function needs and produces.
- **Enables independent development** — as long as the interface is fixed, the function's internal implementation can be changed freely without breaking the calling code.
- **Supports libraries** — header files expose only interfaces (prototypes) of library functions; the actual implementation is compiled separately.

What is a prototype?

A prototype is the one-line declaration of a function that tells the compiler its name, its return type, and the types of arguments it takes — without showing the actual code inside the function. For example, `int add(int a, int b);` says: "there is a function called `add`, it takes two ints, and it gives back an int" — without revealing how the addition actually happens inside.

Communication Between Caller and Function

The interface is the **only channel of communication** between a calling function and the called function. Data flows into the function through **arguments**, and a result flows back through the **return statement**. Everything inside the function body (local variables, algorithm steps) is hidden from the caller — this hiding of internal detail is called **information hiding** or **encapsulation** of logic.

KEY IDEA

"The interface tells you *what* to give and *what* you get back; the function body decides *how* it computes the result."

Benefits in Program Design

Benefit	Explanation
Modularity	Each function becomes an independent, swappable unit connected only via its interface.
Reusability	A well-designed interface allows the same function to be reused in different programs.
Maintainability	Internal changes to a function do not affect other parts of the program, as long as the interface stays the same.
Testability	Functions can be tested in isolation by supplying inputs through the interface and checking the output.
Team collaboration	Programmers can agree on interfaces first, then work independently on implementations.

Example — Interface stays the same, implementation changes

```
// Version 1
int cube(int x) {
    return x * x * x;
}

// Version 2 (different implementation, same interface)
int cube(int x) {
    int result = 1;
    for (int i = 0; i < 3; i++)
        result *= x;
    return result;
}
// Caller code: cube(4);  -- works identically with either version
```

Output (both versions): `64`

Exam Tip

Remember the phrase: *"An interface specifies WHAT, not HOW."* This single line answers most theory questions on the role/importance of a function interface.

CHAPTER SUMMARY

- The interface is the sole communication channel between caller and function.
- It enables type safety, information hiding, modularity, and independent development.
- A stable interface allows the internal implementation to change freely.

TOPIC 04

Arguments of a Subprogram

Definition

DEFINITION

Arguments of a subprogram are the actual data values or variables passed into a subprogram (such as a function, procedure, method, or subroutine) when it is called. They allow the subprogram to receive inputs, execute its logic using those specific data points, and vary its behaviour every time it runs.

While often used interchangeably in casual conversation, computer science maintains a strict technical distinction between "arguments" and "parameters" — arguments are the values supplied at the call site, while parameters are the placeholders that receive them in the subprogram's definition (see Actual and Formal Arguments below).

Actual and Formal Arguments

Aspect	Actual Arguments	Formal Arguments
Where used	In the function **call**	In the function **definition/header**
What they are	Real values, variables, or expressions supplied by the caller	Placeholder variables that receive the actual argument's value
Scope	Belong to the calling function	Belong to the called function (local to it)
Example	`add(5, 6);` — 5 and 6 are actual arguments	`int add(int a, int b)` — a and b are formal arguments

Example — Actual vs formal arguments

```
int add(int a, int b) {     // a, b -> FORMAL arguments
    return a + b;
}
int main() {
    int x = 5, y = 6;
    int sum = add(x, y);    // x, y -> ACTUAL arguments
    printf("Sum = %d\n", sum);
    return 0;
}
```

Output: `Sum = 11`

Parameter Passing

When a function is called, the actual arguments are matched with the formal arguments **in order and by type**. C primarily uses two parameter passing mechanisms:

- **Call by value** — a copy of the actual argument's value is passed (default in C; see Topic 13).
- **Call by reference (simulated using pointers)** — the address of the actual argument is passed, letting the function modify the caller's variable (see Topic 14).

Note

The number, order, and type of actual arguments in a function call must match the number, order, and type of formal arguments in the function definition/prototype — this is enforced by the compiler using the function's interface.

Example — Functions with multiple arguments

```
float average(float a, float b, float c) {
    return (a + b + c) / 3;
}
int main() {
    printf("Average = %.2f\n", average(4.0, 5.0, 9.0));
```

```
    return 0;
}
```

Output: `Average = 6.00`

Common Mistake

Passing arguments in the wrong order (e.g., swapping width and height in an area function) compiles fine if types match, but produces logically wrong results. Always match argument *order*, not just type.

CHAPTER SUMMARY

- Arguments let a function operate on different data across different calls.
- Actual arguments appear in the call; formal arguments appear in the function definition.
- Arguments are matched by position and type; C mainly uses call by value and (via pointers) call by reference.

TOPIC 05

Kinds of Subprograms Supported by C

DEFINITION

C classifies subprograms along two independent lines: **where they come from** (library functions vs. user-defined functions) and **how they communicate** (with or without arguments, with or without a return value). Together, these classifications describe every kind of subprogram C supports.

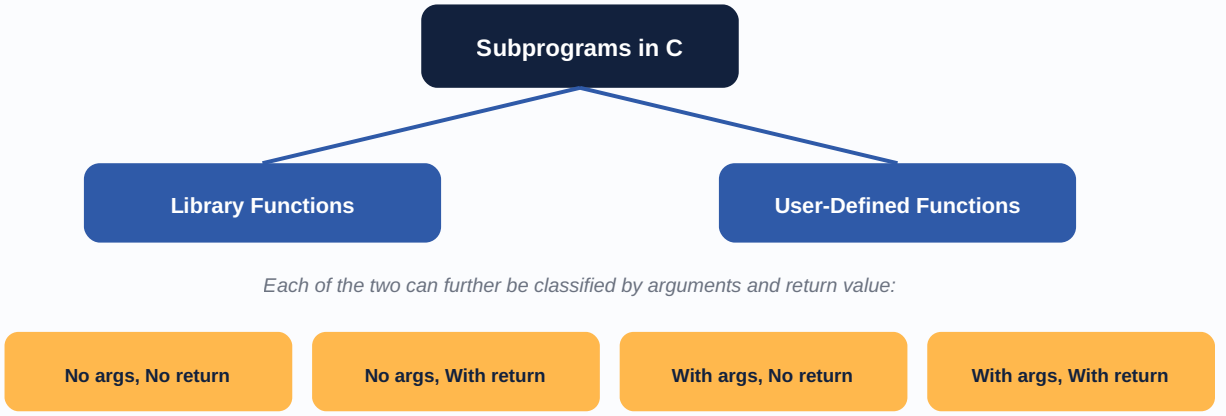


Diagram: Classification of subprograms supported by C

Library Functions

DEFINITION

Library functions are pre-written, pre-compiled functions provided by the C standard library (e.g., in `stdio.h`, `math.h`, `string.h`). They are ready to use simply by including the appropriate header file.

Example — Library functions

```
#include <stdio.h>
#include <math.h>
int main() {
    printf("Square root = %.2f\n", sqrt(25.0));  // library function
    return 0;
}
```

Output: `Square root = 5.00`

User-Defined Functions

DEFINITION

User-defined functions are functions written by the programmer to perform tasks specific to a program's requirements.

Example — User-defined function

```
#include <stdio.h>
int factorial(int n) {          // user-defined function
    int f = 1;
    for (int i = 1; i <= n; i++) f *= i;
    return f;
```

```c
}
int main() {
    printf("Factorial = %d\n", factorial(5));  // printf -> library function, factorial -> user-defined function
    return 0;
}
```

Output: `Factorial = 120`

Exam Tip

Library functions save development time and are tested/optimized by the compiler vendor; user-defined functions are needed whenever the required logic is not already available in the standard library.

Functions With / Without Arguments

Type	Description	Example call
Without arguments	Does not receive any data from the caller.	`greet();`
With arguments	Receives one or more values from the caller.	`greet("Aman");`

Functions With / Without Return Values

Type	Description	Return type
Without return value	Performs an action but sends nothing back.	`void`
With return value	Computes and sends a result back to the caller.	`int`, `float`, etc.

Comparison Table — All Four Combinations

Category	Example prototype	Behaviour
No arguments, no return value	`void greet(void);`	Just performs an action (e.g., prints a message).
No arguments, with return value	`int getValue(void);`	Produces a result without needing input (e.g., reads and returns user input).
With arguments, no return value	`void display(int n);`	Uses the given data but does not send anything back.
With arguments, with return value	`int add(int a, int b);`	Most common form — takes input and produces output.

Example — All four combinations in one program

```c
#include <stdio.h>

void greet(void) {              // no args, no return value
    printf("Hello!\n");
}
int getValue(void) {            // no args, with return value
    return 100;
}
void display(int n) {           // with args, no return value
    printf("Value = %d\n", n);
}
int add(int a, int b) {         // with args, with return value
    return a + b;
}

int main() {
    greet();
    printf("%d\n", getValue());
    display(25);
    printf("Sum = %d\n", add(4, 6));
```

```
    return 0;
}
```

Output: `Hello! 100 Value = 25 Sum = 10`

Exam Tip

A commonly asked MCQ: "Which category needs neither the caller to send nor receive data?" Answer: **No arguments, no return value**.

CHAPTER SUMMARY

- C supports library functions (predefined, from the standard library) and user-defined functions (written by the programmer).
- Functions may or may not take arguments, and may or may not return a value — giving four possible combinations.
- The four combinations are: no args/no return, no args/with return, with args/no return, and with args/with return — the last being the most common.

TOPIC 06

Return Statement as an Interface

Purpose of the Return Statement

DEFINITION

The `return` statement is the part of a function's interface that sends a value back to the calling function and simultaneously ends the function's execution, transferring control back to the point of call.

SYNTAX

```
return expression;   // returns a value and exits the function
return;              // exits a void function (no value)
```

Returning Values From Functions

- The type of the returned expression should match the function's declared return type.
- A function may contain multiple `return` statements, but only **one** of them executes per call.
- As soon as a `return` statement executes, control leaves the function immediately — any statements after it (in the same path) are skipped.

Example — Single return statement

```
int square(int x) {
    return x * x;    // sends result back, exits function
}
int main() {
    printf("%d\n", square(7));
    return 0;
}
```

Output: `49`

Multiple Return Scenarios

Example — Multiple return statements (conditional)

```
char grade(int marks) {
    if (marks >= 90) return 'A';
    else if (marks >= 75) return 'B';
    else if (marks >= 50) return 'C';
    else return 'F';
}
int main() {
    printf("Grade: %c\n", grade(82));
    return 0;
}
```

Output: `Grade: B`

Note

Only one `return` statement executes in any single function call — "multiple return statements" refers to multiple *possible* exit points in the code, not multiple values returned in one call. C does not support returning several values directly (use a struct, array, or pointer arguments to return multiple results).

Common Mistake

Forgetting a `return` statement in a non-void function leads to undefined behaviour — the function may appear to "return" a garbage value.

CHAPTER SUMMARY

- `return` both sends a value back and ends the function's execution.
- A function can have several return statements on different execution paths, but only one runs per call.
- Returning multiple values directly is not possible; structs/pointers/arrays are used instead.

TOPIC 07

Local Variables

Definition

DEFINITION

A **local variable** is a variable declared inside a function (or a block) that is accessible *only* within that function/block, and exists only while that function/block is executing.

Scope

- A local variable's **scope** (visibility) is limited to the function or block in which it is declared.
- It cannot be accessed from outside that function, including from `main()` or other functions.
- Variables with the same name in different functions are treated as completely independent variables.

Lifetime

- By default, local variables have **automatic storage duration** — memory for them is allocated when the function is called and released when the function returns.
- Their value does **not persist** (it doesn't get saved or remembered) between separate calls to the function (unless declared `static`).

Example — Local variable scope

```
void funcA() {
    int x = 10;      // local to funcA
    printf("In A: x = %d\n", x);
}
void funcB() {
    int x = 99;      // different variable, local to funcB
    printf("In B: x = %d\n", x);
}
int main() {
    funcA();
    funcB();
    return 0;
}
```

Output: `In A: x = 10 In B: x = 99`

Example — Lifetime: value resets on every call

```
void counter() {
    int count = 0;      // reinitialized every call
    count++;
    printf("Count = %d\n", count);
}
int main() {
    counter();
    counter();
    counter();
    return 0;
}
```

Output: `Count = 1 Count = 1 Count = 1`

Memory Representation

Local variables are stored in **stack memory**. Each function call creates a new **stack frame (activation record)** that holds its own local variables, formal parameters, and the return address. When the function returns, its stack frame is destroyed and the memory is reclaimed.

Call stack after main() calls funcA(), which calls inner()

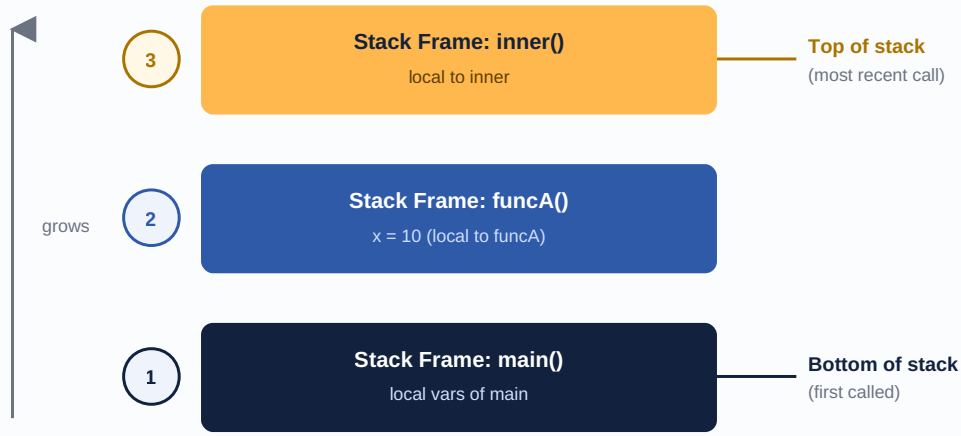


Diagram: Call stack — numbers show push order (1 → 3); frames pop in reverse (3 → 1) as functions return

What is stack memory?

Stack memory is a reserved area of memory that a program uses to store local variables and manage function calls. It works like a stack of plates: the most recently added item is the first one removed (LIFO — Last In, First Out). When a function is called, its local variables are "pushed" onto the stack; when the function returns, they are automatically "popped off" and their memory is freed — which is exactly why local variables don't persist between calls.

Exam Tip

Prefix `static` to a local variable to make it retain its value between calls (e.g., `static int count = 0;`) — its *scope* still stays local, only its *lifetime* changes to the whole program run.

CHAPTER SUMMARY

- Local variables are declared inside a function/block and are visible only there.
- They are created on function entry and destroyed on function exit (stack memory), unless static.
- Each function call gets its own independent copy of its local variables.

TOPIC 08

Default Return Type and the void Type

Default Return Type

DEFINITION

In old (pre-ANSI / K&R) C, if a function's return type was not explicitly specified, the compiler assumed it to be `int` by default. This is called the **implicit int rule**.

Example — Old-style implicit int (not recommended)

```
square(int x) {          // no return type written -> assumed int
    return x * x;
}
```

Common Mistake

Modern C standards (C99 and later) have **removed** the implicit-int rule. Always specify an explicit return type for every function — relying on the default is now considered bad practice and may cause compilation errors/warnings in modern compilers.

The void Return Type

DEFINITION

`void` as a return type indicates that the function **does not return any value** to the caller.

SYNTAX

```
void functionName(parameter_list) {
    // statements
    // optional: return;   (exits early, no value)
}
```

Example — void return type

```
void printMessage(void) {
    printf("Hello from a void function!\n");
}
int main() {
    printMessage();
    return 0;
}
```

Output: `Hello from a void function!`

void Parameters

Writing `void` inside the parentheses of a function explicitly states that the function takes **no arguments**.

Example — void parameter list

```
int getRandomNumber(void) {   // explicitly: no parameters
    return 42;
}
```

Note

In C (unlike C++), an **empty** parameter list `func()` means "unspecified arguments" (no type-checking on arguments), while `func(void)` explicitly means "zero arguments" and is fully type-checked. Best practice: always use `void` for functions that take no parameters.

Comparison Table

Aspect	int (default/explicit)	void
Meaning as return type	Function returns an integer value	Function returns nothing
Use of `return expr;`	Mandatory (must return an int)	Not allowed; only bare `return;` permitted
Meaning as parameter	N/A	Function takes zero arguments
Can store in a variable?	Yes — `int r = func();`	No — calling code cannot assign the call to a variable

CHAPTER SUMMARY

- Old C assumed `int` as the default return type if omitted; modern C requires an explicit return type.
- `void` as a return type means the function returns nothing.
- `void` as a parameter list means the function accepts nothing.

TOPIC 09

Passing Values Between Functions Through Interfaces

Data Passing Mechanism

Data moves between the caller and a function strictly through the function's interface: **arguments carry data in**, and the **return statement carries a result out**. C provides two conceptual mechanisms for this exchange:

Mechanism	Direction	How
Call by value	Input only (one-way, into the function)	A copy of the value is passed; changes inside the function do not affect the caller's variable.
Call by reference (via pointers)	Input and output (two-way)	The address of the variable is passed, so the function can read and modify the caller's original variable.
Return statement	Output only (one-way, out of the function)	A single value is sent back to the point of call.

Input and Output Through Parameters

- **Input parameters** — supply data to the function (used with call by value, or a pointer used only for reading).
- **Output parameters** — pointers passed so the function can write results back into the caller's variables.
- **Input/output parameters** — pointers used both to read the current value and write back an updated value.

Short examples — Input, Output, and Input/Output parameters

```
// INPUT parameter: only supplies data, nothing written back
int square(int n) {          // n is input-only
    return n * n;
}

// OUTPUT parameter: pointer used only to write a result back
void getSquare(int n, int *result) {
    *result = n * n;         // writes back to caller, n is not modified
}

// INPUT/OUTPUT parameter: pointer used to read AND write
void doubleIt(int *n) {
    *n = *n * 2;             // reads current value, then updates it
}
```

Function Communication

KEY IDEA

A function is like a "black box": data enters through its parameter interface, is processed internally, and a result exits through the return statement (or through pointer parameters for multiple outputs).



Flowchart: Data entering and leaving a function through its interface

Example — Output via return value

```
int square(int n) {
    return n * n;         // one value out
}
```

Example — Multiple outputs via pointer parameters

```
void minMax(int a, int b, int *min, int *max) {
    if (a < b) { *min = a; *max = b; }
    else       { *min = b; *max = a; }
}
int main() {
    int lo, hi;
    minMax(15, 8, &lo, &hi);
    printf("Min = %d, Max = %d\n", lo, hi);
    return 0;
}
```

Output: `Min = 8, Max = 15`

Exam Tip

Since a C function can `return` only a single value, pointer parameters are the standard technique to "return" multiple results — this is a very frequently asked practical/viva question.

CHAPTER SUMMARY

- Data enters a function via arguments and leaves via the return statement (or pointer parameters).
- Call by value gives one-way input; pointer arguments allow two-way communication.
- Pointer parameters are the standard way to return multiple values from a C function.

TOPIC 10

Declaration of Function Type

Function Declaration (Prototype)

DEFINITION

A **function declaration** tells the compiler about a function's name, return type, and parameter types *before* it is actually used, without providing the function body.

SYNTAX

```
return_type function_name(parameter_type_list);
```

Example

```
int add(int a, int b);   // declaration / prototype
```

Function Definition

DEFINITION

A **function definition** provides the actual body/implementation of the function — the statements that execute when it is called.

SYNTAX

```
return_type function_name(parameter_list) {
    // body: statements
    return value;   // if not void
}
```

Example

```
int add(int a, int b) {   // definition
    return a + b;
}
```

Function Call

DEFINITION

A **function call** is a statement that invokes/executes a function, optionally passing arguments and using its returned value.

SYNTAX

```
function_name(actual_argument_list);
```

Example — Declaration, definition, and call together

```
#include <stdio.h>

int add(int a, int b);      // 1. DECLARATION (prototype)

int main() {
    int result = add(3, 4); // 3. CALL
    printf("%d\n", result);
    return 0;
}

int add(int a, int b) {     // 2. DEFINITION
```

```
    return a + b;
}
```

Output: 7

Syntax and Examples — Summary Table

Stage	Contains body?	Ends with	Typical location
Declaration	No	Semicolon `;`	Before `main()` or in a header file
Definition	Yes	Closing brace `}`	Anywhere in the file (often after `main()`)
Call	N/A	Semicolon `;`	Inside another function's body

Note

If a function is **defined before** it is used (e.g., above `main()`), a separate declaration is not strictly required, because the definition itself serves as the declaration.

CHAPTER SUMMARY

- Declaration = interface only (no body); Definition = full implementation; Call = actual invocation.
- A function must be declared or defined before it is called.

TOPIC 11

Iterative and Recursive Subprograms

Iterative Subprogram

DEFINITION

A function that solves a problem by repeating a set of statements using loops (`for` , `while` , `do-while`) until a condition is met.

Working mechanism: Iteration uses a single stack frame; the loop variable is updated repeatedly within the same function call.

Example — Factorial: Iterative version

```
int factorialIter(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++)
        result *= i;
    return result;
}
// Call: factorialIter(5);
```

Output: `120`

Recursive Subprogram

DEFINITION

A function that solves a problem by **calling itself**, with a smaller version of the same problem, until a base condition is reached.

Working mechanism: Recursion creates a **new stack frame for every call**, and unwinds (returns) the calls once the base case is hit.

Example — Factorial: Recursive version

```
int factorialRec(int n) {
    if (n == 0) return 1;            // base case
    return n * factorialRec(n - 1);  // recursive case
}
// Call: factorialRec(5);
```

Output: `120`

Advantages and Disadvantages

Aspect	Iteration	Recursion
Speed	Generally faster (no function-call overhead)	Slower (overhead of repeated function calls)
Memory usage	Low, constant (single stack frame reused)	Higher — proportional to recursion depth (multiple stack frames)
Code clarity	Can become complex for naturally recursive problems (e.g., tree traversal)	Often simpler and closer to the mathematical definition
Risk	Infinite loop if condition never becomes false	Stack overflow if base case is missing/unreachable
Best suited for	Simple repetitive tasks (sum, search, sorting with loops)	Problems with recursive structure (tree/graph traversal, divide-and-conquer, factorial, Fibonacci)

Common Mistake

Writing a recursive function without a correctly reachable base case causes **infinite recursion**, which eventually crashes the program with a **stack overflow** error.

Exam Tip

Every recursive solution can be rewritten iteratively using an explicit stack, and vice versa — a frequently asked theory point.

CHAPTER SUMMARY

- Iteration repeats using loops within one stack frame; recursion repeats by self-calls, each with its own stack frame.
- Iteration is generally faster and uses less memory; recursion is often simpler for naturally recursive problems.
- Both must have a proper terminating condition to avoid infinite execution.

TOPIC 12

Recursion

Concept of Recursion

DEFINITION

Recursion is a programming technique in which a function calls itself, directly or indirectly, to solve a problem by breaking it into smaller instances of the same problem.

Base Case and Recursive Case

- **Base case** — the condition under which the function stops calling itself and returns a direct answer (prevents infinite recursion).
- **Recursive case** — the part where the function calls itself with a smaller/simpler input, moving the problem closer to the base case.

SYNTAX

```
return_type recursiveFunc(parameters) {
    if (base_condition)
        return base_result;         // base case
    else
        return recursiveFunc(smaller_input);  // recursive case
}
```

Recursive Function Execution / Call Stack

Every recursive call is pushed onto the **call stack** as a new stack frame containing its own copy of parameters and local variables. Once the base case is reached, the calls begin to **return** (unwind) one by one, each using the result of the call below it, until control reaches back to the original caller.

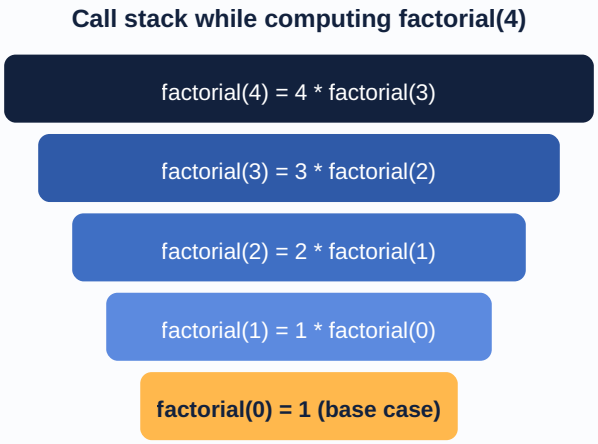


Diagram: Recursive function call stack for factorial(4)

Examples

Example 1 — Factorial using recursion

```
int factorial(int n) {
    if (n == 0)                 // base case
        return 1;
    return n * factorial(n - 1); // recursive case
```

```
}
int main() {
    printf("%d\n", factorial(4));
    return 0;
}
```

Output: `24`

Example 2 — Fibonacci series using recursion

```
int fibonacci(int n) {
    if (n == 0) return 0;         // base case 1
    if (n == 1) return 1;         // base case 2
    return fibonacci(n - 1) + fibonacci(n - 2); // recursive case
}
int main() {
    for (int i = 0; i < 6; i++)
        printf("%d ", fibonacci(i));
    return 0;
}
```

Output: `0 1 1 2 3 5`

Example 3 — Sum of first n natural numbers

```
int sum(int n) {
    if (n == 0) return 0;
    return n + sum(n - 1);
}
// Call: sum(5);
```

Output: `15`

Common Mistake

Missing or unreachable base case → infinite recursion → **stack overflow**. Always verify that each recursive call moves the argument strictly closer to the base case.

Exam Tip

Plain recursive Fibonacci has exponential time complexity $O(2^n)$ because of repeated recalculation of the same sub-values — a frequently tested comparison point against iteration or memoized recursion.

CHAPTER SUMMARY

- Recursion = a function calling itself with a smaller input, guided by a base case and a recursive case.
- Each call adds a new frame to the call stack; frames unwind once the base case is reached.
- Classic examples: factorial, Fibonacci, sum of n numbers.

TOPIC 13

Concept of Call by Value

Definition

DEFINITION

In **call by value**, a **copy** of the actual argument's value is passed to the function's formal parameter. This is the **default** parameter-passing mechanism in C.

Working Mechanism

- When the function is called, the value of each actual argument is copied into the corresponding formal parameter.
- The formal parameter is a completely separate variable, stored at a different memory location from the actual argument.
- Any changes made to the formal parameter inside the function affect only the **copy**; the original variable in the caller remains unchanged.

Memory Representation



Two independent memory locations: changing x inside modify() does NOT change a inside main()

Diagram: Call by value — memory representation (separate copies)

Example — Call by value (variable unchanged in caller)

```
void modify(int x) {
    x = x + 10;             // changes only the local copy
    printf("Inside modify: x = %d\n", x);
}
int main() {
    int a = 5;
    modify(a);
    printf("Inside main: a = %d\n", a);   // unaffected
    return 0;
}
```

Output: `Inside modify: x = 15 Inside main: a = 5`

Advantages and Limitations

Advantages	Limitations
Original data is safe from accidental modification.	Cannot modify the caller's original variables.

Simple to understand and use.	Copying large data (e.g., big structs) repeatedly can be inefficient.
Each call is independent; no side effects on caller's data.	Cannot directly return multiple values this way.

Exam Tip

A classic exam trap: writing a "swap two numbers" function using plain call by value fails to swap the caller's variables — this is exactly why call by reference (Topic 14) is needed.

CHAPTER SUMMARY

- Call by value passes a copy of the argument; it is the default mechanism in C.
- Changes inside the function do not affect the original variable in the caller.
- Safe but cannot be used to modify caller data or return multiple values.

TOPIC 14

Concept of Call by Reference

Definition

DEFINITION

In **call by reference**, instead of the value, the **address (memory location)** of the actual argument is passed to the function. C does not have true call-by-reference syntax like some other languages — it is **simulated using pointers**.

Passing Addresses Using Pointers

SYNTAX

```
void functionName(int *ptr) {
    *ptr = new_value;     // dereference to modify original variable
}
// Call:
functionName(&variable);   // pass address using &
```

Working Mechanism

- The `&` (address-of) operator is used at the call site to pass a variable's address.
- The function's formal parameter is declared as a **pointer** that stores this address.
- Inside the function, the `*` (dereference) operator is used to access/modify the value at that address — which is the caller's original variable.

Memory Representation



*ptr does not hold its own copy of the data — it points directly to a's memory location, so *ptr = 50 changes a*

Diagram: Call by reference (simulated with pointers) — memory representation

Example — Call by reference (variable changed in caller)

```
void modify(int *x) {
    *x = *x + 10;          // modifies the caller's original variable
    printf("Inside modify: *x = %d\n", *x);
}
int main() {
    int a = 5;
    modify(&a);
    printf("Inside main: a = %d\n", a);   // changed!
```

```
    return 0;
}
```

Output: `Inside modify: *x = 15 Inside main: a = 15`

Example — Swapping two numbers using call by reference

```
void swap(int *p, int *q) {
    int temp = *p;
    *p = *q;
    *q = temp;
}
int main() {
    int a = 3, b = 8;
    swap(&a, &b);
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

Output: `a = 8, b = 3`

Advantages and Limitations

Advantages	Limitations
Can modify the caller's original variables directly.	More error-prone — incorrect pointer use can cause bugs/crashes.
Efficient for large data (no copying; only an address is passed).	Requires careful use of `&` and `*` operators.
Allows a function to effectively "return" multiple values.	Risk of dereferencing invalid/NULL pointers (undefined behaviour).

Common Mistake

Forgetting `&` at the call site, or `*` when accessing the value inside the function, are the two most common pointer-related errors in call-by-reference code.

CHAPTER SUMMARY

- Call by reference passes an address (via a pointer), not a copy of the value.
- Changes made using the pointer directly affect the caller's original variable.
- It is used when a function must modify caller data or "return" multiple values.

TOPIC 15

Return Mechanism and Underlying Implementation

Function Call Process

When a function is called, the following steps occur internally, in order:

1. The actual arguments are evaluated.
2. A new **stack frame (activation record)** is created for the called function.
3. The values of arguments are copied into the formal parameters (or addresses, for pointer parameters) inside this new frame.
4. The **return address** (the point in the calling function where execution should resume) is saved.
5. Control jumps to the first statement of the called function.
6. The function body executes, using its own local variables (also stored in its stack frame).
7. On reaching a `return` statement (or the closing brace), the return value (if any) is placed in a designated location (typically a CPU register), the function's stack frame is popped/destroyed, and control returns to the saved return address.

Stack Memory

> **KEY IDEA**
>
> Function calls are managed using a **call stack** — a Last-In-First-Out (LIFO) data structure. Each function call pushes a new frame; each return pops the top frame.

Each stack frame typically stores:

- Formal parameters (or their copies)
- Local variables of that function call
- The return address (where to resume in the caller)
- Saved register values (to be restored after the call)

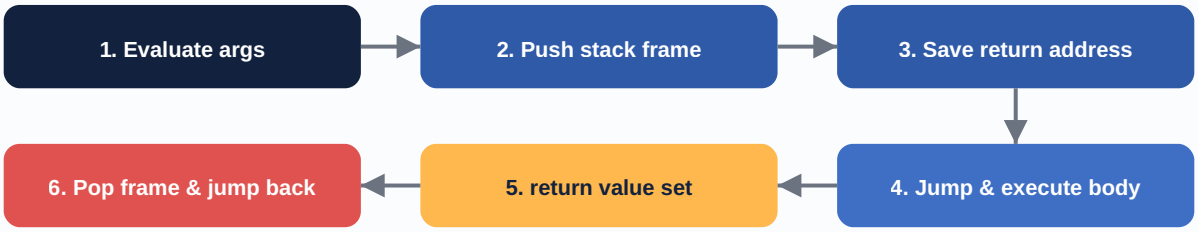
Parameter Passing (Recap in Implementation Terms)

Mechanism	What's stored in the new stack frame
Call by value	A copy of the actual argument's value
Call by reference (pointer)	A copy of the actual argument's *address*, which points back to the original memory location

Returning Control to the Calling Function

On function exit, three things happen in order:

1. **Return value is transferred** to the caller — often via a CPU register such as EAX/RAX in many implementations, or copied directly for larger types.
2. **The called function's stack frame is deallocated**, freeing its local variables' memory.
3. **Execution resumes** at the saved return address in the calling function, continuing from right after the call.



Control returns to caller, execution resumes right after the call

This entire cycle repeats for every nested/recursive function call

Diagram: Function call and return process (underlying implementation)

Exam Tip

Remember the six-step cycle: *Evaluate arguments → Push frame → Save return address → Execute body → Set return value → Pop frame & jump back*. This sequence is a very common short-answer/viva question.

CHAPTER SUMMARY

- Function calls are implemented using a LIFO call stack of activation records (stack frames).
- Each frame holds parameters, local variables, and the return address.
- On return, the frame is destroyed, the value is passed back, and control resumes after the call.

TOPIC 16

Similarities and Comparisons

Call by Value vs Call by Reference

Similarities

- Both are mechanisms for passing arguments into a function.
- Both are evaluated at the moment the function is called.
- Both are ultimately implemented in C as *call by value* at the mechanical level — call by reference simply passes a copy of an *address* (a pointer) rather than a copy of the data itself.

Aspect	Call by Value	Call by Reference
What is passed	Copy of the value	Address of the variable (pointer)
Effect on original variable	No effect	Can be modified
Memory usage	Extra memory for the copy	Only the address is stored (efficient for large data)
Operators used	None special	`&` (address-of), `*` (dereference)
Safety	Safer — original data protected	Riskier — original data can be unintentionally changed
Use case	When the function should not alter caller data	When the function must modify caller data or return multiple values

Iteration vs Recursion

Similarities

- Both are techniques for repeating a set of instructions.
- Both need a proper terminating condition to avoid running forever.
- Any problem solved with one can also be solved with the other.

Aspect	Iteration	Recursion
Mechanism	Loops (for/while/do-while)	Function calling itself
Termination	Loop condition becomes false	Base case is reached
Memory	Single stack frame (constant)	Multiple stack frames (proportional to depth)
Speed	Generally faster	Generally slower (call overhead)
Risk	Infinite loop	Stack overflow
Code style	Can be verbose for recursive problems	Often more concise for recursively-structured problems

Function Declaration vs Definition vs Function Call

Similarities

- All three always refer to the *same* function.
- They must agree on its name, parameter types, and return type.
- A mismatch between any of them is a compile-time error.

Aspect	Declaration	Definition	Call
Purpose	Informs compiler of the interface	Provides the implementation	Invokes/executes the function

Body present?	No	Yes	N/A
Occurs	Before use, or in header file	Once, anywhere in the program	Wherever the function is needed
Ends with	`;`	`{ ... }`	`;`

User-Defined vs Library Functions

Similarities

- Both are called using the same function-call syntax.
- Both are governed by a defined interface (name, parameters, return type) that the caller must respect.
- Both can accept arguments and return values.

Aspect	User-Defined Functions	Library Functions
Who writes it	The programmer	Provided by the C standard library
Need for header file?	No (written in the same/linked file)	Yes — `#include` the relevant header
Source visible?	Yes, fully visible/editable	No, only the interface (prototype) is visible; implementation is precompiled
Example	`int add(int a, int b)`	`sqrt()`, `printf()`, `strlen()`

Exam Tip

Comparison-table questions are extremely common in university and competitive exams — memorize at least 3–4 distinguishing points for each pair above.

CHAPTER SUMMARY

- Call by value protects caller data; call by reference allows modification via pointers.
- Iteration uses loops with constant memory; recursion uses self-calls with stack-proportional memory.
- Declaration, definition, and call are three distinct stages of using a function.
- User-defined functions are written by the programmer; library functions come precompiled with the standard library.

TOPIC 17

Programming Examples

1. Function With No Arguments / No Return Value

Program — Print a welcome message

```
#include <stdio.h>
void welcome() {
    printf("Welcome to C Functions!\n");
}
int main() {
    welcome();
    return 0;
}
```

Output: `Welcome to C Functions!`

Explanation: `welcome()` takes no input and sends nothing back — it simply performs an action (printing a message).

2. Function With Arguments / No Return Value

Program — Display a number's square

```
#include <stdio.h>
void displaySquare(int n) {
    printf("Square of %d is %d\n", n, n * n);
}
int main() {
    displaySquare(6);
    return 0;
}
```

Output: `Square of 6 is 36`

Explanation: The value 6 is received as an argument and used, but the function does not send any value back — its return type is `void`.

3. Function With Arguments and Return Value

Program — Find the greater of two numbers

```
#include <stdio.h>
int greater(int a, int b) {
    if (a > b) return a;
    return b;
}
int main() {
    printf("Greater = %d\n", greater(23, 47));
    return 0;
}
```

Output: `Greater = 47`

Explanation: Two values are passed in as arguments, and one computed value is sent back via `return` — the most commonly used function form.

4. Recursive Program

Program — Sum of digits of a number using recursion

```c
#include <stdio.h>
int sumDigits(int n) {
    if (n == 0) return 0;              // base case
    return (n % 10) + sumDigits(n / 10); // recursive case
}
int main() {
    printf("Sum of digits = %d\n", sumDigits(1234));
    return 0;
}
```

Output: `Sum of digits = 10`

Explanation: Each call extracts the last digit and recurses on the remaining number until `n` becomes 0 (the base case), then the digit sums accumulate as the calls unwind.

5. Program Demonstrating Call by Value

Program — Attempt to double a number (call by value)

```c
#include <stdio.h>
void doubleValue(int n) {
    n = n * 2;
    printf("Inside function: n = %d\n", n);
}
int main() {
    int num = 10;
    doubleValue(num);
    printf("Inside main: num = %d\n", num);
    return 0;
}
```

Output: `Inside function: n = 20 Inside main: num = 10`

Explanation: Since only a copy of `num` is passed, doubling it inside the function has no effect on `num` in `main()`.

6. Program Demonstrating Call by Reference

Program — Double a number using call by reference

```c
#include <stdio.h>
void doubleValue(int *n) {
    *n = *n * 2;
    printf("Inside function: *n = %d\n", *n);
}
int main() {
    int num = 10;
    doubleValue(&num);
    printf("Inside main: num = %d\n", num);
    return 0;
}
```

Output: `Inside function: *n = 20 Inside main: num = 20`

Explanation: Passing the address of `num` lets the function modify it directly through the pointer, so the change is visible in `main()` as well.

> **Exam Tip**
> In practical/lab exams, you are often asked to write the **same logic twice** — once using call by value and once using call by reference — to demonstrate the difference in output. Practice this pairing for functions like swap, increment, and doubling.

CHAPTER SUMMARY

- All four argument/return-value combinations were demonstrated with working programs.
- A recursive program (sum of digits) illustrated base case and recursive case in action.
- Call by value vs call by reference was demonstrated side-by-side using the same "double a number" logic.

TOPIC 18

Quick Revision

Important Definitions (One-Line Recap)

Term	One-line definition
Subprogram / Function	Named, reusable block of code performing a specific task.
Interface	Function name + parameter types + return type — the contract with the caller.
Argument	Value passed to a function; *actual* in the call, *formal* in the definition.
Local variable	Variable declared inside a function; visible and alive only during that call.
void	Indicates "returns nothing" or "takes no parameters".
Recursion	A function calling itself with a smaller input, guided by a base case.
Call by value	Passes a copy of the value; caller's variable unaffected.
Call by reference	Passes an address (pointer); caller's variable can be modified.
Stack frame	Memory block holding a function call's parameters, locals, and return address.

Syntax Summary

FUNCTION PROTOTYPE

```
return_type function_name(parameter_type_list);
```

FUNCTION DEFINITION

```
return_type function_name(parameter_list) {
    // body
    return value;   // omit if void
}
```

FUNCTION CALL

```
function_name(actual_argument_list);
```

CALL BY REFERENCE

```
void func(int *p) { *p = value; }
func(&variable);
```

RECURSIVE FUNCTION

```
return_type func(params) {
    if (base_condition) return base_value;
    return func(smaller_input);
}
```

Function Prototype Format

FORMAT

`return_type function_name(parameter_type_1, parameter_type_2, ...);` — parameter *names* are optional in a prototype; only *types* are mandatory.

Frequently Asked Exam Questions

- What is a subprogram? What are its advantages?
- Explain the interface of a function with a diagram.
- Differentiate between actual and formal arguments.

- What are library and user-defined functions? Give examples.
- Explain the role of the `return` statement.
- What is the scope and lifetime of a local variable?
- Differentiate between `void` as a return type and as a parameter.
- Explain how values are passed between functions.
- Differentiate between function declaration, definition, and call.
- Compare iterative and recursive subprograms.
- Explain recursion with the base case and recursive case, using an example.
- Differentiate between call by value and call by reference with examples.
- Explain the underlying implementation of a function call and return using the stack.
- Write a C program to swap two numbers using call by reference.
- Write a recursive C program to compute the factorial of a number.

One-Page Revision Notes

Functions in one glance

Functions decompose a program into reusable, testable units. Interface = name + params + return type. Arguments carry data in; `return` carries a result out. Local variables live in stack frames and vanish after the call ends (unless static).

Parameter passing in one glance

Call by value → copy, caller safe, cannot modify. Call by reference (pointers) → address passed, caller's data can be changed, needed for multiple outputs and efficiency with large data.

Recursion in one glance

Base case stops the calls; recursive case reduces the problem each time. Each call = new stack frame; results combine as calls unwind. Watch for missing base cases → stack overflow.

Call mechanics in one glance

Call sequence: evaluate args → push frame → save return address → execute → set return value → pop frame → resume in caller. This LIFO cycle underlies both iteration-free calls and deeply nested recursive calls.

Common Mistakes and Debugging Tips

Mistake	Fix / Tip
Missing return statement in a non-void function	Ensure every code path returns a value of the correct type.
Mismatched argument types/order in a call	Always check the function prototype before calling.
Expecting call by value to modify the caller's variable	Use call by reference (pointers) when modification is needed.
Forgetting `&` or `*` with pointers	Remember: `&` gets an address, `*` dereferences it.
Recursive function without a reachable base case	Trace through a small input by hand to confirm the base case is always eventually reached.
Assuming local variables retain values across calls	They don't, unless declared `static`.
Relying on implicit `int` return type	Always write the return type explicitly.

FINAL TAKEAWAY

- A function's interface (name, parameters, return type) governs all communication with its caller.
- Call by value protects data; call by reference (via pointers) allows modification and multi-value "returns".
- Recursion and iteration are two ways to repeat work — recursion via self-calls and the stack, iteration via loops.
- Every function call follows the same underlying stack-based call-and-return mechanism.